

COTEPECOS

COLEGIO TÉCNICO PROFESIONAL

Sistema de Notas Estudiantiles

II Etapa — Lógica y estructuras de datos (Bucles, Arreglos y Matrices)

Proyecto de Programación · Continuación del Sistema de Notas diseñado en la I Etapa

Estudiante: Damian Quirós Jiménez

Sección: 11-5

Subárea: Programación Web

Nivel: 11°

Profesor(a): _____

Fecha de entrega: 13 y 14 de octubre, 2026

Índice

1. Introducción
2. Bucles recorriendo el sistema
3. Arreglos en el sistema
4. Matriz de notas (estudiantes $\times$ cursos)
5. Integración de las estructuras con la POO
6. Código completo: cálculo del promedio general
7. Conclusión

Introducción

En la I Etapa se diseñaron cuatro clases (`Estudiante` , `Curso` , `Nota` y `Profesor`) explicando qué guarda cada una y cómo se relacionan. El problema es que, en un colegio real, no hay un solo estudiante: hay treinta, cuarenta, o los que tenga la sección. Crear una variable distinta para cada uno (`Estudiante e1` , `Estudiante e2` , `Estudiante e3` ...) no es manejable.

Para eso sirven los **bucles** y los **arreglos**: un arreglo guarda muchos objetos del mismo tipo bajo un solo nombre, y un bucle permite recorrerlos a todos sin repetir código. Cuando la información tiene dos dimensiones —por ejemplo, varios estudiantes con varias notas cada uno, una por curso— se usa una **matriz**, que es un arreglo de arreglos.

Este documento retoma exactamente las clases de la I Etapa y muestra cómo esas estructuras de datos se apoyan en ellas para procesar información real del sistema.

1. Bucles recorriendo el sistema

Indicador 1

Un bucle sirve para repetir una acción sobre cada elemento de una colección sin escribir esa acción una y otra vez a mano. En Java se puede usar `for` (cuando se sabe cuántas veces hay que repetir, por ejemplo "una vez por cada estudiante del arreglo") o `while` (cuando se repite mientras se cumpla una condición). Para recorrer un arreglo completo, lo normal es usar `for` , porque el propio arreglo ya sabe su tamaño (`.length`).

Ejemplo descriptivo

El sistema puede tener un arreglo con los estudiantes matriculados en un curso. Un bucle `for` recorre ese arreglo desde el primero hasta el último estudiante, y en cada vuelta le pide que muestre sus datos. En palabras simples, el bucle "pasa revista" a todos los estudiantes uno por uno, sin que haya que escribir una línea distinta por cada uno.

Ejemplo de código

```
Estudiante[] listaEstudiantes = new Estudiante[3];
listaEstudiantes[0] = new Estudiante("118760123", "Damian Quirós", "damian@correo.com");
listaEstudiantes[1] = new Estudiante("205430987", "Ana Rojas", "ana@correo.com");
listaEstudiantes[2] = new Estudiante("109980456", "Luis Mora", "luis@correo.com");

for (int i = 0; i < listaEstudiantes.length; i++) {
    listaEstudiantes[i].mostrarDatos();
}
```

Ejemplo de explicación

Se declara un arreglo de objetos `Estudiante` llamado `listaEstudiantes` , con espacio para 3 posiciones. Cada posición se llena con un objeto distinto usando `new` . El bucle `for` usa un contador `i` que arranca en

0 y sube de uno en uno mientras sea menor que `listaEstudiantes.length` (en este caso, 3). En cada vuelta, `i` apunta a un estudiante distinto del arreglo, y se llama a su método `mostrarDatos()`. Así se procesan los tres estudiantes con las mismas 3 líneas de código, sin importar si el arreglo tuviera 3 o 300 estudiantes.

La misma idea se puede escribir con `while`, controlando el contador a mano:

```
int i = 0;
while (i < listaEstudiantes.length) {
    listaEstudiantes[i].mostrarDatos();
    i++;
}
```

Diferencia clave: con `for` el contador nace, se compara y se incrementa en una sola línea, así que es más difícil olvidar el `i++`. Con `while` hay que acordarse de subir `i` manualmente adentro del bucle, o el programa nunca termina (bucle infinito).

2. Arreglos en el sistema

Indicador 2

Un arreglo (*array*) es un bloque de casillas numeradas, todas del mismo tipo, guardadas bajo un solo nombre de variable. En este sistema, en vez de guardar cada `Estudiante` o cada `Nota` en una variable suelta, se guardan varios dentro de un arreglo de ese tipo.

Ejemplo descriptivo

Un mismo estudiante saca varias notas a lo largo del curso: un examen, un proyecto, un quiz, una tarea. En vez de tener cuatro variables sueltas (`nota1`, `nota2` ...), esas cuatro notas se guardan en un arreglo de objetos `Nota`, todas agrupadas bajo un solo nombre.

Ejemplo de código

```
Nota[] notasDeUnEstudiante = new Nota[4];
notasDeUnEstudiante[0] = new Nota(88.5, "Examen", "14/10/2026");
notasDeUnEstudiante[1] = new Nota(92.0, "Proyecto", "20/10/2026");
notasDeUnEstudiante[2] = new Nota(75.0, "Quiz", "28/10/2026");
notasDeUnEstudiante[3] = new Nota(80.0, "Tarea", "02/11/2026");

for (int i = 0; i < notasDeUnEstudiante.length; i++) {
    notasDeUnEstudiante[i].mostrarNota();
}
```

Ejemplo de explicación

`notasDeUnEstudiante` es un arreglo de tamaño fijo (4 posiciones, definidas al crearlo con `new Nota[4]`). A diferencia de guardar solo números sueltos, cada posición guarda un objeto `Nota` completo, con su valor, su tipo de evaluación y su fecha —porque esos tres datos son justamente los atributos privados que se

definieron para la clase `Nota` en la I Etapa. El bucle `for` recorre el arreglo igual que en el punto anterior, y por cada posición pide el detalle con `mostrarNota()`.

3. Matriz de notas (estudiantes × cursos)

Indicador 3

Una matriz es, en el fondo, un arreglo de arreglos: una tabla con filas y columnas. Para este sistema tiene sentido una matriz donde **cada fila es un estudiante** y **cada columna es un curso**, de modo que la posición `[fila]` `[columna]` guarda la nota final de ese estudiante en ese curso.

	Programación (col. 0)	Matemática (col. 1)	Español (col. 2)
Damian (fila 0)	88.5	92.0	79.0
Ana (fila 1)	95.0	68.0	84.0
Luis (fila 2)	70.0	73.5	90.0

Ejemplo descriptivo

Para leer una matriz completa hace falta un bucle dentro de otro bucle (bucles anidados): el bucle de afuera se mueve fila por fila (estudiante por estudiante), y por cada fila, el bucle de adentro recorre columna por columna (curso por curso) antes de pasar a la siguiente fila.

Ejemplo de código

```
double[][] matrizNotas = {
    { 88.5, 92.0, 79.0 }, // fila 0: notas de Damian
    { 95.0, 68.0, 84.0 }, // fila 1: notas de Ana
    { 70.0, 73.5, 90.0 }  // fila 2: notas de Luis
};

for (int fila = 0; fila < matrizNotas.length; fila++) {
    for (int columna = 0; columna < matrizNotas[fila].length; columna++) {
        System.out.println("Estudiante " + fila + " - Curso " + columna
            + ": " + matrizNotas[fila][columna]);
    }
}
```

Ejemplo de explicación

`matrizNotas` es un arreglo de arreglos `double`: cada elemento de `matrizNotas` no es un número, sino otro arreglo (una fila completa). `matrizNotas.length` da la cantidad de filas (3 estudiantes). Adentro del bucle externo, `matrizNotas[fila].length` da la cantidad de columnas de esa fila específica (3 cursos). El bucle externo (`fila`) avanza estudiante por estudiante; por cada estudiante, el bucle interno (`columna`) recorre sus notas de cada curso, una por una, antes de que el bucle externo pase al siguiente estudiante.

4. Integración de las estructuras con la POO

Indicador 4

Los arreglos y las matrices no reemplazan a las clases de la I Etapa: las **organizan en grupo**. Un arreglo `Estudiante[]` sigue guardando objetos `Estudiante` completos, con todos sus atributos privados y su encapsulamiento intactos; el arreglo solo agrega la capacidad de tener muchos a la vez y recorrerlos con un bucle.

Esto conecta directamente con el diagrama de clases de la I Etapa: la relación "*Curso matricula 0..* Estudiante*" que se dibujó ahí, en código, se puede representar exactamente con un arreglo. La clase `Curso` podría tener un atributo `private Estudiante[] matriculados;` para guardar a todos sus estudiantes, y un método que los recorra con un `for` para, por ejemplo, calcular cuántos aprobaron. De la misma forma, la relación "*Estudiante posee 0..* Nota*" se puede representar con un arreglo `Nota[]` dentro de `Estudiante`.

La matriz del punto 3 funciona igual, pero en vez de guardar solo el número de la nota (`double`), también podría ser una matriz de objetos completos (`Nota[][] matrizNotas`), donde cada casilla guarda un objeto `Nota` con su valor, tipo de evaluación y fecha, en vez de un simple decimal. La versión con `double[][]` que se usó arriba es más simple y suficiente para calcular promedios, que es justamente lo que se necesita en el siguiente punto.

En resumen: las estructuras de datos (arreglos, matrices) son el "contenedor" que agrupa muchos objetos de las clases diseñadas en la I Etapa; los bucles son la herramienta para recorrer ese contenedor y hacer algo con cada objeto que hay adentro.

5. Código completo: cálculo del promedio general

Indicador 5

Este último punto junta todo lo anterior: un arreglo de `Estudiante`, una matriz de notas, y bucles anidados para calcular el promedio general de cada estudiante y guardarlo de vuelta en su objeto, usando el `setPromedio()` definido en la I Etapa.

Ejemplo descriptivo

Por cada estudiante (fila de la matriz) se necesita sumar todas sus notas (columnas de esa fila) y dividir esa suma entre la cantidad de cursos, para obtener el promedio general. Eso es exactamente un bucle externo (por estudiante) con un bucle interno (por curso) que va acumulando una suma.

Ejemplo de código

```
public class Main {
    public static void main(String[] args) {

        Estudiante[] estudiantes = {
            new Estudiante("118760123", "Damian Quirós", "damian@correo.com"),
            new Estudiante("205430987", "Ana Rojas", "ana@correo.com"),
            new Estudiante("109980456", "Luis Mora", "luis@correo.com")
        };

        double[][] notas = {
            { 88.5, 92.0, 79.0 },
            { 95.0, 68.0, 84.0 },
            { 70.0, 73.5, 90.0 }
        };

        for (int i = 0; i < estudiantes.length; i++) {

            double suma = 0;
            for (int j = 0; j < notas[i].length; j++) {
                suma = suma + notas[i][j];
            }

            double promedioGeneral = suma / notas[i].length;
            estudiantes[i].setPromedio(promedioGeneral);
            estudiantes[i].mostrarDatos();
        }
    }
}
```

Ejemplo de explicación

El arreglo `estudiantes` y la matriz `notas` están alineados a propósito: la fila `i` de `notas` corresponde siempre al estudiante en la posición `i` de `estudiantes` (por eso Damian está en la posición 0 de ambos, Ana en la 1, Luis en la 2). El bucle externo (`i`) avanza estudiante por estudiante. Por cada estudiante, la variable `suma` se reinicia en 0, y el bucle interno (`j`) recorre todas las notas de esa fila, acumulándolas en

`suma` . Cuando el bucle interno termina, `suma` tiene el total de las notas de ese estudiante; dividiéndolo entre `notas[i].length` (la cantidad de cursos) se obtiene el promedio general. Ese promedio no se queda suelto: se guarda dentro del objeto con `estudiantes[i].setPromedio(promedioGeneral)` , respetando el encapsulamiento definido en la I Etapa —nada fuera de la clase toca el atributo `promedio` directamente. Por último, `mostrarDatos()` imprime el resultado ya guardado.

Conclusión

Esta etapa no cambió ni una clase de las que se diseñaron en la I Etapa: `Estudiante`, `Curso`, `Nota` y `Profesor` siguen siendo exactamente las mismas, con sus atributos privados y sus métodos `get / set` intactos. Lo que se agregó fue la forma de **manejar muchos objetos de esas clases a la vez**: arreglos para guardarlos en grupo, matrices para organizar información en dos dimensiones (estudiantes × cursos), y bucles —simples y anidados— para recorrer todo eso sin repetir código.

El código del punto 5 es, en la práctica, el motor que le falta al sistema para que sea útil de verdad: a partir de una lista de estudiantes y una matriz de notas, calcula y guarda el promedio general de cada uno usando exactamente los métodos que ya existían. Esa misma base de arreglos y bucles es la que se va a usar en la III Etapa para poblar la interfaz gráfica con los datos del sistema.